

Chapter 3 – System Design

3.1 Introduction

This chapter presents the system design of the Document Tracking System Software, which has been architected using the Model-View-Controller (MVC) design pattern. The system design plays a crucial role in bridging the gap between the requirements gathered and the actual implementation of the software. It outlines how the core components of the system interact, and provides a blueprint for developers to follow during development and testing.

The proposed system is designed as a modular and scalable web application, ensuring maintainability and ease of enhancement. The MVC architecture was chosen due to its clear separation of concerns, allowing the system to be divided into three main components:

Model – Manages data and business logic related to document tracking, including database operations for storing, updating, and retrieving document metadata, status, and movement logs.

View – Responsible for rendering the user interface and displaying document status, reports, and alerts. It ensures a user-friendly experience for various roles such as administrators, document handlers, and supervisors.

Controller – Acts as an intermediary between the Model and the View. It handles user inputs, processes requests, invokes appropriate business logic, and updates the View accordingly.

This design chapter will detail the system architecture, module decomposition, database design, and interface design. Diagrams such as the system architecture diagram, use case diagrams, class diagrams, and data flow diagrams will be presented to illustrate the internal structure and flow of the system.

The goal of this chapter is to ensure that the overall system structure is logically sound and can support the required functionalities such as document uploading, tracking by barcode, user access control, and report generation.

A typical document tracking system architecture is a multi-tiered, modular design that includes a user interface layer, a core application layer (with modules for document management, workflow, user authentication, and tracking), a data storage layer for documents and metadata, and potentially an integrations layer for connecting with other business systems. Key architectural considerations include scalability, flexibility, support for version control, robust security, comprehensive backup and disaster recovery, and workflow automation.

3.1 Project Objectives

Even though the system can be integrated with the other sections of the institute, the selected research area is the finance section, which handles all the financial works such as course payments management of the postgraduate students.

1. To Use bar code to the file and read it
2. To secure the received files
3. To Retrieving lost copies and storing a copy separately
4. To file share
5. To admin Provide another person with the ability to view files other than
6. To Use of encryption for storing files
7. User friendly interface
8. To save money and time for the customer
9. To an update every day

To give a unique password

3.1.1 Functional Objectives

Develop a system that automates the tracking of physical documents, reducing reliance on manual methods. Enable real-time monitoring of document locations and movements within an organization.

Create a centralized platform where authorized personnel can access documents in real-time. Support remote collaboration through secure web-based access.

Minimize time to spend on manual tracking of the documents by automating these processes.

Allow employees to focus on higher-value tasks by reducing administrative burdens.

Assess the system in terms of user acceptability, reliability, maintainability, scalability, and security. Gather feedback from end-users to refine features and improve overall usability.

- **User Management and Authentication:**

- **User Registration:** Allow new users (e.g., administrative staff, students, auditors) to register with unique credentials.
- **User Login/Logout:** Provide a secure authentication mechanism for users to log in and out of the system.
- **Role-Based Access Control (RBAC):** Implement distinct user roles (e.g., Administrator, Finance Staff, Postgraduate Student) with varying levels of permissions for viewing, adding, editing, and deleting documents and system functionalities.
 - **Administrator:** Full control over users, roles, departments, document types, system settings.
 - **Finance Staff:** Ability to upload, categorize, update status, and search for finance-related documents.
 - **Postgraduate Student:** Ability to upload their own finance documents, view the status of their submitted documents, and potentially view relevant announcements.

- **Auditor (optional):** Read-only access to specific financial documents and audit trails.
 - **Password Management:** Enable users to securely reset forgotten passwords and change existing ones.
 - **User Profile Management:** Allow users to view and update their profile information.
-
- **Document Management:**
 - **Document Upload:** Allow authorized users to upload various document types (e.g., payment receipts, scholarship applications, fee waivers) in supported formats (e.g., PDF, JPEG, PNG).
 - **Document Categorization:** Enable the classification of documents using:
 - **Document Type:** (e.g., "Course Fee Payment Receipt," "Research Grant Application," "Refund Request Form").
 - **Department/Section:** (e.g., "Financial Section," "Student Affairs").
 - **Student ID/Name:** Link documents directly to specific postgraduate students.
 - **Academic Year/Semester:** For easy historical retrieval.
 - **Document Metadata Input:** Allow users to input essential descriptive information for each document, such as:
 - Document Title/Description
 - Submission Date and Time
 - Student ID and Name
 - Associated Financial Amount (for financial documents)
 - Current Status (e.g., "Submitted," "Pending Approval," "Approved," "Rejected," "Processed")
 - Unique Reference/Transaction Number
 - **Document Storage:** Securely store digital copies of documents in an organized manner.
 - **Document Viewing/Download:** Allow authorized users to securely view and download uploaded documents.
 - **Document Editing/Updating:** Allow authorized users (e.g., Finance Staff) to modify document metadata and status.
 - **Document Version Control (Optional but Recommended):** Track different versions of a document if revisions or updates are common, allowing users to view previous versions.
 - **Document Deletion (Admin Only):** Provide a secure mechanism for authorized administrators to permanently delete documents, accompanied by an audit trail.
 - **Document Tracking and Workflow:**
 - **Status Updates:** Allow Finance Staff to update the processing status of each document (e.g., from "Submitted" to "Pending Verification" to "Approved for Payment" to "Processed").
 - **Automated Notifications:** Implement automatic notifications (e.g., email alerts, in-system notifications) to relevant users upon:

- Successful document submission by a student.
 - Change in document status.
 - Request for additional information from a student or staff.
 - Document reaching a critical stage in the workflow.
- **Audit Trail/Activity Log:** Maintain a detailed, immutable log of all actions performed on each document (who performed the action, what action was performed, when it was performed, and from where). This is crucial for accountability, transparency, and auditing.
- **Basic Workflow Management:** Define and visualize a simple, configurable workflow for financial documents (e.g., Student Submits -> Finance Staff Verifies -> Head of Finance Approves -> Payment Processed).
- **Search and Reporting:**
 - **Advanced Search:** Provide powerful search capabilities allowing users to find documents based on:
 - Keywords (in title, description, or potentially OCR'd content of documents)
 - Document Type
 - Student ID/Name
 - Submission Date Range
 - Current Document Status
 - Department/Section
 - Financial Amount Range
 - **Filtering and Sorting:** Allow users to filter search results and sort them by various criteria (e.g., date, student ID, status).
 - **Customizable Reports:** Generate predefined and potentially customizable reports on:
 - Number of documents by type, status, or department.
 - Documents pending approval/processing (with age analysis).
 - Historical document activity and trends.
 - List of documents by a specific student.
 - Overdue documents (if a processing timeline is implemented).
 - **Export Functionality:** Allow exporting report data to common formats (e.g., CSV, Excel, PDF).

3.1.2 Non-Functional objectives

Performance - Ensure the system responds quickly to user actions, such as document searches or updates, with a response time of less than 2 seconds under normal load.

Scalability - Design the system to handle increasing numbers of users and documents without performance degradation.

Security - Implement robust encryption for data storage and transmission to prevent unauthorized access.

Reliability - Ensure the system operates consistently with minimal downtime, achieving at least 99.9% availability. Incorporate fault-tolerant mechanisms to recover quickly from failures.

Usability - Provide an intuitive and user-friendly interface that simplifies navigation and minimizes training requirements.

3.2 Related design strategies

Architecture with Security First Put in place a multi-layered security model. This covers data level security (encryption), application level security (secure coding techniques), and network level security (firewalls, intrusion detection).

Separation of Concerns: Make sure that the database, application logic (back-end), and web front-end are all clearly separated in the system design. This reduces the extent of a possible breach and avoids a single point of failure.

Centralized Repository: Make use of a safe, central repository for documents. In addition to preventing data fragmentation, this "single source of truth" facilitates the application and enforcement of uniform security policies, such as access controls and audit trails, across all documents.

Data Security Strategies

Encryption: This is fundamental. **Encryption at Rest:** Encrypt all documents stored in the database or file system. This ensures that even if a malicious actor gains access to the physical storage, the data remains unreadable without the decryption key.

Encryption in Transit: Use protocols like HTTPS (TLS/SSL) to encrypt all data transmitted between the user's browser and the server. This prevents man-in-the-middle attacks where data could be intercepted as it travels over the network.

Data Classification: Categorize documents by sensitivity (e.g., public, internal, confidential, top secret). This allows for a more granular and efficient application of security policies, ensuring that the most sensitive documents receive the highest level of protection without unnecessarily restricting access to less critical information.

Authentication and Access Control

Role-Based Access Control (RBAC): Secure document systems are built on this foundation. Give users roles (such as administrator, manager, or employee) and then give each role particular permissions

(such as view, edit, delete, and share). This lowers the possibility of human error and streamlines the administration of user permissions.

Strong Authentication: To stop unwanted access, use strong user authentication.

Using multi-factor authentication (MFA), users must supply two or more verification factors in order to be granted access. In addition to a basic password, this adds a crucial layer of security.

Secure Password Policies: Require frequent password changes and enforce strict password length and complexity guidelines.

Session Management: To stop session hijacking, use secure session management. Using secure cookies and configuring suitable session timeouts are two examples of this.

A fundamental security principle is the principle of least privilege. Create the system with the bare minimum of access and permissions required for users and processes to carry out their designated tasks.

Monitoring, Auditing, and Accountability

Entire Audit Trails: The main purpose of a document tracking system is to keep track of documents. Strong audit trails that record each action are essential to a secure system. This contains:

Who had access to a document?

What modifications were made?

When the action occurred.

From where the action originated (e.g., IP address).

Version Control: Automatically track and store different versions of a document as it is edited. This provides a history of changes, prevents data loss, and allows for the recovery of previous versions.

Digital Signatures: Incorporate digital signatures to verify the authenticity

- **Digital Signatures:** Incorporate digital signatures to verify the authenticity and integrity of a document. This ensures that a document has not been tampered with since it was signed.
- **Real-time Notifications:** Implement a notification system to alert users of document updates, access requests, or other relevant events.

Other Key Design Strategies

- **User Experience (UX) and Usability:** A secure system is useless if users can't or won't use it. Design an intuitive and user-friendly interface that makes it easy for users to follow security protocols without hindering their productivity.
- **Compliance:** Design the system to comply with relevant industry regulations and standards (e.g., HIPAA for healthcare, GDPR for data privacy).
- **Scalability:** The system should be able to scale to handle a growing number of users, documents, and tracking events without compromising performance or security.
- **Disaster Recovery and Backups:** Implement a regular and automated backup strategy. Backups should be stored securely in multiple locations (e.g., on-site and off-site/cloud) to protect against data loss from hardware failure, natural disasters, or cyberattacks.
- **Regular Security Audits and Penetration Testing:** After development, it's crucial to perform regular security audits and penetration tests to identify and fix vulnerabilities before they can be exploited by malicious actors.

3.3 System Architecture

A well-designed system architecture is the blueprint for a secure document tracking system. It defines how all the components work together to deliver the required functionality while ensuring data security and integrity. A robust architecture for this type of system typically follows a multi-layered approach to provide defense-in-depth.

Here is a breakdown of the typical system architecture for a web-based secure document tracking system:

1. Presentation Layer (Client-Side)

This is the front-end that users interact with. Its primary role is to provide a user-friendly interface for all document-related activities.

- **Components:**
 - **User Interface (UI):** This is built using technologies like HTML, CSS, and JavaScript. It's the visual part of the application, including dashboards, forms for uploading documents, search fields, and a document viewer.
 - **Web Browser:** The primary client application used to access the system.
 - **Client-Side Scripting:** JavaScript frameworks (e.g., React, Angular, Vue.js) are used to handle user interactions, validate form data, and make API calls to the server without requiring a full page reload.

2. Application Layer (Server-Side)

This is the core of the system where the business logic resides. It handles all the complex processes, requests, and security validations. This layer is crucial for a secure system as it acts as a gatekeeper between the user and the data.

- **Components:**
 - **Web Server:** (e.g., Apache, Nginx) This component receives incoming requests from the client, routes them to the correct application component, and serves static content.
 - **Application Server:** This is where the core business logic is executed. It is built using programming languages like Java (Spring Boot), Python (Django, Flask), or Node.js. Its responsibilities include:
 - **Authentication and Authorization:** Verifying user identity and permissions for every action.
 - **Document Management Logic:** Handling document uploads, versioning, metadata extraction, searching, and sharing.
 - **Document Tracking and Auditing:** Logging all user actions (who, what, when, where).
 - **Encryption/Decryption Engine:** Managing the encryption and decryption of documents at rest.
 - **Workflow Engine:** (Optional but recommended) This component automates document approval processes and notifications.
 - **API Gateway:** This acts as a single entry point for all client requests. It can perform functions like load balancing, security enforcement (e.g., rate limiting), and routing requests to different microservices.

3. Data Layer

This layer is responsible for the secure storage and retrieval of all system data. It is separated from the application layer to enhance security.

- **Components:**
 - **Document Storage:** This is where the actual documents are stored. For enhanced security, documents should be encrypted and stored in a location that is not directly accessible from the web. This could be a secure file system, an object storage service (e.g., Amazon S3, Azure Blob Storage), or a dedicated document management database.
 - **Database:** (e.g., PostgreSQL, MySQL, MongoDB) This relational or NoSQL database stores all the metadata related to the documents, user information, permissions (for RBAC), audit logs, and tracking data.
 - **Database Management System (DBMS):** Manages the data and enforces data integrity rules. It is essential to configure the DBMS with strong security protocols.

4. Security and Infrastructure Layer

This layer underpins the entire system, providing the necessary security measures and infrastructure to ensure its availability and resilience.

- **Components:**
 - **Firewall:** Filters network traffic to prevent unauthorized access.

- **Intrusion Detection/Prevention System (IDS/IPS):** Monitors network and system activities for malicious behavior and can take action to block it.
- **Secure Sockets Layer (SSL)/Transport Layer Security (TLS):** This is a critical protocol for encrypting all data in transit between the client and the server (e.g., using HTTPS).
- **Identity and Access Management (IAM):** A dedicated service that manages user identities, authentication, and access policies. It can integrate with enterprise directories like LDAP or Active Directory.
- **Logging and Monitoring:** A centralized logging system to collect all events from different layers. This is essential for security auditing, troubleshooting, and anomaly detection.
- **Backup and Disaster Recovery:** A system for regularly backing up the document storage and database, with a plan for restoring services in case of a disaster.

Architectural Diagram (High-Level View)

A conceptual diagram would show a three-tiered architecture:

- **Tier 1: Client/Presentation Layer:** User's browser making HTTPS requests.
- **Tier 2: Application Layer:** Web Server and Application Server(s) processing requests and business logic.
- **Tier 3: Data Layer:** Document Storage and Database, securely storing encrypted documents and their metadata.

Arrows would show the flow of information:

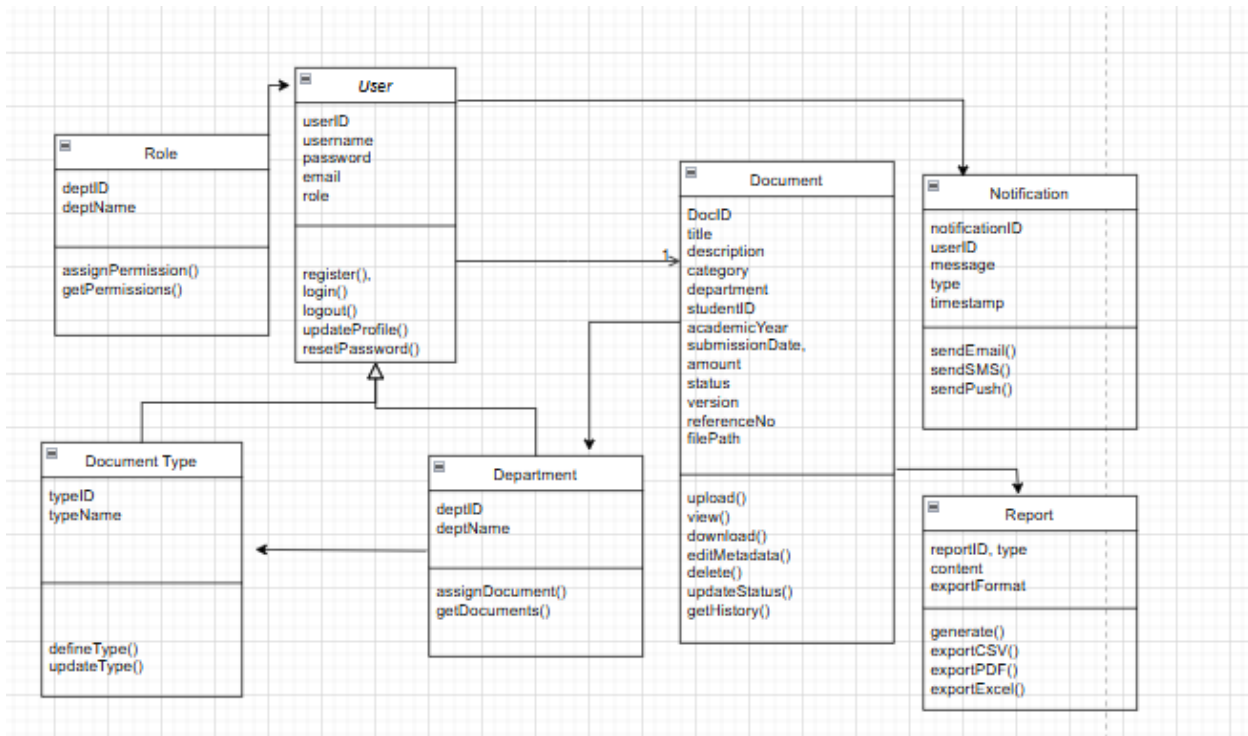
1. User's Browser sends a request to the Web Server via HTTPS.
2. Web Server passes the request to the Application Server.
3. Application Server processes the request (e.g., authenticates the user, checks permissions).
4. Application Server interacts with the Database for metadata and the Document Storage for the encrypted file.
5. Application Server builds the response and sends it back to the Web Server.
6. Web Server sends the response to the User's Browser via HTTPS.

3.4 Use case diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved. This use case diagram shows the identified actors and their related use cases.



User Case Diagram



Class Diagram

Summary

The system architecture for a web-based secure document tracking system is typically designed in a tiered, component-based structure to ensure high security, scalability, and maintainability. It can be understood by looking at four main layers: the Client Layer, the Web Server Layer, the Application Server Layer, and the Data Tier.

List of References

1. Bhutada.S1, Kalluri.M2, Dontha K3,04 | Apr-2018 , "In-house File Tracking System", <https://www.irjet.net/archives/V5/i4/IRJET-V5I4260.pdf>
2. . StanfieldJoseph .J.D, BrancThomas.R, FeldpauschWilliam .G,Miller.L 1995File folders for use in an electronic "file locating and tracking system", <https://patents.google.com/patent/US5739765A/en>
3. Kumar.V, Barwal.P.N,P , Murthy.B.K "File Tracking System "International Journal of

Scientific Engineering and Applied Science (IJSEAS) - Volume-1, Issue-2, May 2015

4. Omoregbe .N.A, Azeta A.A, Adewumi .A, Edeh.E.E Proceedings of EDULEARN14 Conference 7th- 9th July 2014, Barcelona, Spain "DEVELOPMENT OF A FILE TRACKING SYSTEM FOR TERTIARY INSTITUTIONS

"https://www.researchgate.net/publication/308880966_DEVELOPMENT_OF_A_FILE_TRACKING_SYSTEM_FOR_TERTIARY_INSTITUTIONS

5. Bandaya.M.T*, Sheikha.S.A, Ratherb.J.A, -16 -18 March, 2015 "File Tracking System for University of Kashmir"

6. Ajoge, Sanni.N & Ibrahim.H.A, April, 2015, "Document E-Tracker: A Panacea for Efficient Internal Document Management System in Kaduna Polytechnic" https://www.researchgate.net/publication/319490546_Document_ETracker_A_Panacea_for_Efficient_Internal_Document_Management_System_in_Kaduna_Polytechnic

7. MassarTed .A, 1994 "Electronic missing file locatorsystem"

<https://patents.google.com/patent/US5450070A/en>

8. Eugene Foster.A 1991 Coded file locator system,
<https://patents.google.com/patent/US5287414A/en> <https://www.ironmtn.co.za>